



# COMPUTAÇÃO PARALELA

AULA 5



Prof. Leonardo Gomes



## CONVERSA INICIAL

A computação paralela considera múltiplos núcleos de processamento e processadores, compartilhando memória em um mesmo sistema. No entanto, nem sempre é possível simplesmente adicionar ilimitadamente mais e mais processadores em um mesmo sistema. A adição de núcleos de processamento, além de um determinado limite, degrada o desempenho geral do sistema devido à competição pelo barramento. No entanto, além de apenas “paralelizar” o processamento, pode-se também “paralelizar e distribuir” o processamento entre diversos computadores independentes, os quais não compartilham o mesmo espaço de memória, mas se comunicam por outras estratégias, como troca de mensagens. Ao longo desta aula, discutiremos tanto as arquiteturas, quanto as formas de programar códigos paralelos para ambientes distribuídos através do *Open MPI*.

### Objetivos da aula

Ao final desta aula esperamos atingir os seguintes objetivos, os quais serão avaliados ao longo da disciplina da forma indicada.

| Objetivos                                                                 | Avaliação                                  |
|---------------------------------------------------------------------------|--------------------------------------------|
| 1 - Entendimento sobre os principais conceitos de computação distribuída. | Atividade prática e Questões Dissertativas |
| 2 - Desenvolvimento de programas simples <i>Open MPI</i> utilizando e C++ | Atividade prática e Questões Dissertativas |
| 3 - Discussão sobre comunicação de processos em <i>Open MPI</i>           | Atividade prática e Questões Dissertativas |
| 4- Discussão sobre sincronismo em processos <i>Open MPI</i>               | Atividade prática e Questões Dissertativas |



## TEMA 1 – INTRODUÇÃO PARA A COMPUTAÇÃO DISTRIBUÍDA

A computação distribuída é o próximo passo da computação paralela. Inicialmente foi a otimização do *clock* dos processadores. Depois, multiplicamos esses processadores em um mesmo sistema e, agora, combinamos máquinas independentes se comunicando e trabalhando em conjunto para resolução de problemas. As demandas computacionais são cada vez maiores ao passar do tempo, pois a quantidade de imagens, sensores, usuários e dispositivos apenas aumentam, assim como a exigência por respostas em tempo real. E, neste sentido, a forma mais eficiente de atender a esta demanda é através da computação distribuída.

Segundo Andrew Tannenbaum, importante cientista da computação e autor dos principais livros da área de computação distribuída e dos sistemas operacionais, um sistema distribuído é uma coleção de computadores independentes que parecem ao usuário como se fossem um único computador. Em certo nível de abstração, os *clusters* de computadores realmente são tratados como uma máquina individual.

As principais características de um sistema distribuído de computadores são três:

- Operam de forma concorrente;
- Devem poder falhar de forma independente, sem comprometer o sistema;
- Os computadores não dividem um mesmo *clock*.

### 1.1 Principais usos da computação distribuída

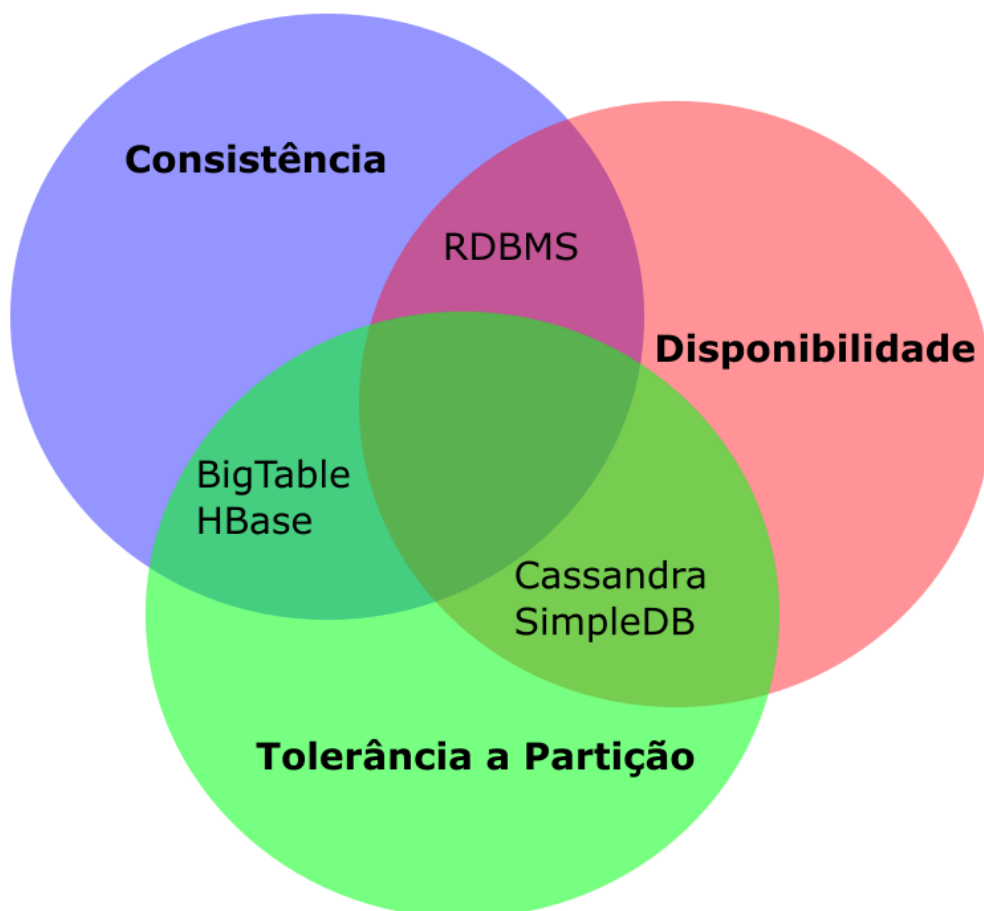
Dentre as principais aplicações da computação distribuída, destacamos três: armazenamento, processamento e troca de mensagens. Na sequência, discutimos as três com mais detalhes.

**Armazenamento:** Bancos de dados possuem grande sinergia com a computação distribuída, dados replicados em diferentes computadores oferecem um grau maior de resiliência em situações de falha ou perda de dados e também promovem uma solução escalonável. Imagine um sistema que, em um curto período de tempo, salte de mil acessos diários para um milhão e, através da adição de computadores extras, o sistema será capaz de responder com a mesma velocidade e aumentar a sua capacidade de armazenamento.



No entanto, os desafios tecnológicos do armazenamento distribuído de dados não são fáceis de lidar. Temos o teorema CAP, sigla para Consistência, Disponibilidade (*Availability*, do inglês) e Partição, que dita que um banco de dados distribuído só pode possuir no máximo duas dessas três características. Consistência: significa que a cada leitura de dado temos a certeza de receber o valor mais atual daquele dado; Disponibilidade: todas as solicitações são respondidas sem erros; Partição: o banco de dados continua funcionando mesmo que uma máquina falhe e o sistema seja particionado. A tolerância a falhas é comumente escolhida, visto que não temos uma forma de garantir que o sistema não enfrentará uma falha, sobrando escolher entre **consistência**, que garante que quando parte do sistema falhar, receberemos uma mensagem de erro sobre eventuais dados que estavam na parte faltante do sistema, ou entre **disponibilidade** a resposta será dada mesmo que desatualizada.

Figura 1 - Ilustração do Teorema CAP e alguns exemplos de bancos de dados que suportam as propriedades indicadas



**Processamento** será o foco do nosso estudo nesta aula. Consiste em distribuir uma grande demanda computacional entre diferentes computadores e



obter uma resposta adequada em um tempo reduzido tanto quanto possível, proporcional ao número de processadores adicionais. Imagine o processamento necessário para o motor de busca da Google pesquisar todos os seus registros em todos os seus servidores. Uma forma de solucionar esse problema, segundo a publicação feita pela própria Google (Dean; Ghemawat, 2004), é a utilização de dois métodos, **map** que consiste do mapeamento da atividade entre os diversos computadores independentes, e **reduce** que une todos os dados processados em uma resposta final. *Hadoop* é um dos *frameworks* mais importantes em *software* livre que implementa os métodos de *map* e *reduce*, conforme descrito pela própria Google em uma série de artigos científicos. Outros paradigmas e ideias surgiram desde então como o *framework* **spark**, da Apache, que utiliza métodos chamados por *scatter/gather*, que em linhas gerais são semelhantes ao *Map/Reduce* do *Hadoop* e trabalha com uma variedade maior de modelagem de dados. Temos o **kafka** também da Apache, um *framework* que possui o paradigma de trabalhar com um fluxo de dados na modalidade *streaming*.

**Troca de mensagens** é um conceito importante em alguns paradigmas de comunicação distribuída, como o **Kafka** da Apache. Ele consiste em dividir a comunicação entre produtores, que fornecem dados, consumidores, que recebem os dados, e tópicos, os canais para onde os produtores despejam o conteúdo e onde os consumidores fazem as buscas. Esse paradigma é interessante pois flexibiliza diferentes tipos de comunicação e evita uma série de conflitos, caso produtores e consumidores se comuniquem diretamente.

## TEMA 2 – CONCORRÊNCIA E CONSENSO

Dentro do contexto de computação distribuída, é comum termos uma arquitetura descentralizada. Isso significa não ter um “líder”, que por um lado oferece uma garantia de disponibilidade do serviço, mesmo com a queda de alguma máquina, porém introduz uma série de problemas para que todas as máquinas cheguem em um consenso de que atividade ou decisão tomar diante de alguma situação em particular.

Alguns exemplos simples de aplicação que podemos citar:

- A sincronização do relógio: algumas máquinas podem ter a informação errada por alguma falha ou por atrasos e latência elevada na rede pode



transmitir uma informação desatualizada. A informação que for consenso na rede entre a maioria das máquinas idealmente deve prevalecer.

- *Blockchain*, as carteiras virtuais das criptomoedas são validadas por meio de um sistema de consenso também. Quanto mais máquinas identificarem uma transação como verdadeira, aumenta a confiança na legitimidade da identidade dos usuários e da transação. Os algoritmos devem ser especialmente robustos neste cenário para garantir usuários mal-intencionados não realizem fraudes se passando por outros, por exemplo.

Um desafio clássico da computação distribuída no contexto de consenso é o problema dos generais bizantinos.

É dito que durante as Cruzadas, legiões do exército bizantino cercavam cidades muradas e coordenavam ataques e retiradas simultaneamente para maior efetividade. Cada general individualmente deseja atacar ou recuar, no entanto uma retirada parcial ou ataque parcial divide o exército comprometendo as forças bizantinas. Cada general controla uma determinada quantidade de tenentes, todo o tipo de comunicação que envolve tanto generais como tenentes, especialmente os que estão distantes entre si, pode ser lenta e sujeita a falhas e, para piorar a situação, é possível que alguns generais ou tenentes sejam traidores que propositalmente dão instruções equivocadas buscando dividir o exército.

Suponha a situação em que temos nove generais ao todo, sendo que quatro generais votem ataque e os outros quatro votem recuar. Sendo o último general malicioso, ele poderia votar para recuar junto ao grupo que decidiu recuar e atacar junto ao grupo que decidiu atacar. Dessa forma, cada grupo se dividiria.

O problema dos generais bizantinos pode ser reduzido para o problema de um general e tenentes. O general dá a instrução para seus tenentes e, caso o general seja malicioso, transmitirá instruções diferentes para cada tenente. E caso um tenente seja malicioso, ele confirmará a instrução com os demais tenentes ao contrário da instrução recebida original.

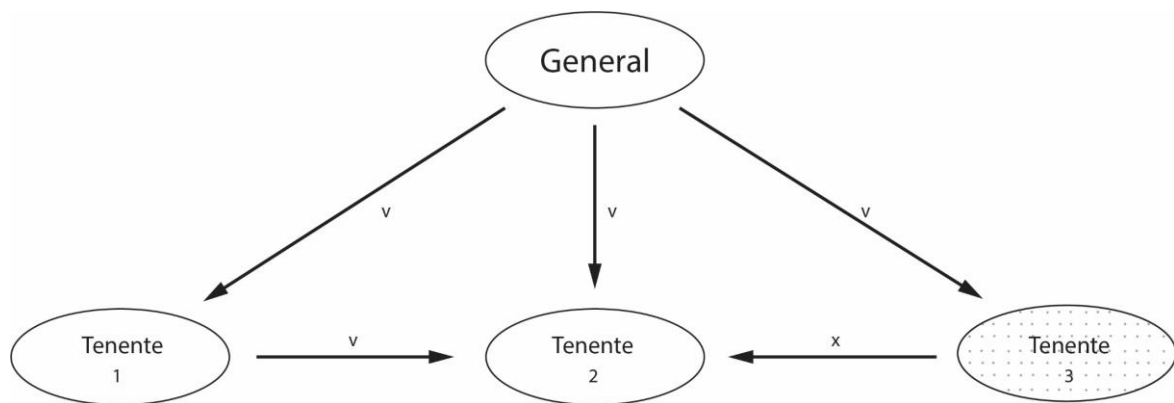
Generais e tenentes são aqui analogias para diferentes grupos de computadores que elegem suas lideranças (generais). As soluções propostas na literatura acadêmica hoje apontam que para  $n$  máquinas maliciosas, são



necessárias  $3n + 1$  máquinas leais para ter uma garantia de que os dados não serão corrompidos.

Vamos analisar a situação de um tenente traidor, na figura 2 abaixo. Existem as instruções **v**, **x** e **z** para decidirem em consenso qual instrução adotar. Cada tenente tem a própria informação recebida do general e as informações repassadas pelos seus colegas tenentes. Neste caso, os tenentes 1 e 2 têm a informação (v, v, x), sendo que v concorda com a informação recebida do seu general, isso mostra que é a maioria e eles executarão a informação v e assumem que o tenente 3 é traidor.

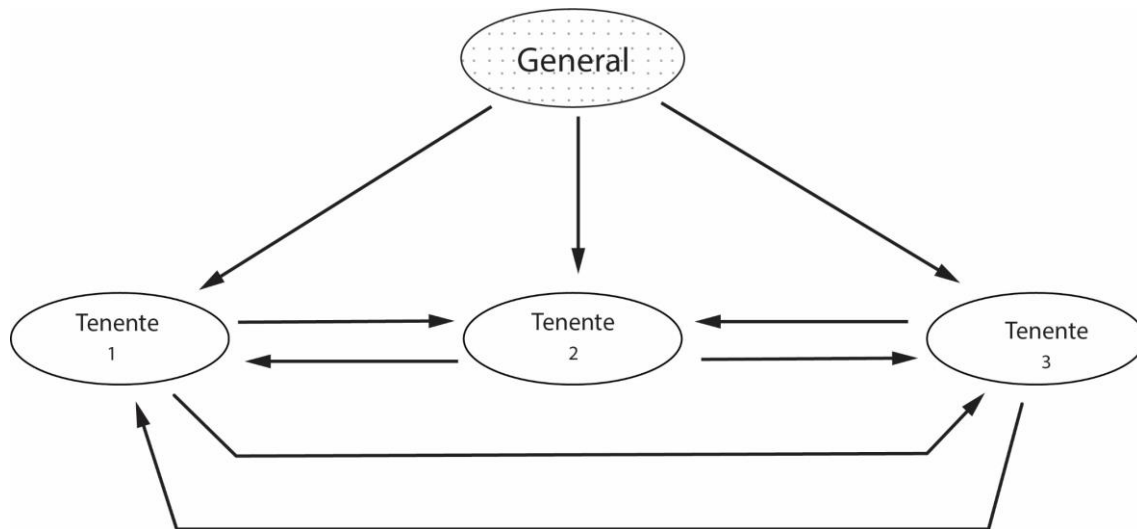
**Figura 2** - Caso de um tenente traidor



Fonte: Lamport, L. et al., 1982

Neste outro cenário, o general é traidor, pois transmite uma informação diferente para cada tenente, que verão as instruções (x, y, z). Sabendo que a instrução recebida não combina com a informação recebida do general, então cada tenente assume que o general deve ser o traidor e adota uma instrução padrão. A instrução padrão na analogia seria recuar ao invés de atacar, visto que a hora do ataque informada pelo general deve estar errada.

Figura 3 - Caso de um general traidor



Fonte: Lamport, L. et al, 1982.

### TEMA 3 – OPEN MPI

O Open MPI é um projeto de *software* livre que implementa o modelo de comunicação *Message Passing Interface* - MPI (em tradução livre, significa interface de passagem de mensagem). Este modelo permite de forma rápida a troca de mensagens entre múltiplos processos, rodando em múltiplas máquinas no intuito de paralelizar o processamento. Veremos como dar os primeiros passos nesta tecnologia ao longo deste tema.

#### 3.1 Visão geral

*Message Passing Interface* (MPI) é um padrão de comunicação que surgiu em 1994 e apresenta uma forma de executar código em paralelo. O MPI Forum é o grupo originalmente formado por mais de 60 profissionais na área que definem estes padrões. Originalmente compatível com C, Fortran e C++, porém desde a versão 3.0 C++ não é mais suportado.

MPI é pensado para funcionar sobre um sistema paralelo de memória distribuída e através de um paradigma de mensagens possui rotinas que oferecem:

- Controle de processos;
- Distribuição e controle do fluxo de dados entre vários computadores;
- Sincronismo.





Com uma camada de alto nível de abstração, o modelo MPI promove uma grande portabilidade e escalabilidade. A solução desenvolvida para rodar em paralelo em duas máquinas pode rodar em 50 máquinas sem retrabalho do programador.

Embora o MPI seja uma especificação, existem diversas implementações disponíveis hoje. Desde os proprietários HP MPI, Intel MPI, MATLAB MPI até versões *software* livre como MPICH e Open MPI.

Através de um grupo formado por pesquisadores, membros da academia e indústria que combinaram suas expertises na intenção de gerar e manter a melhor biblioteca que implementa os padrões MPI que surgiu a *Open MPI*.

Vamos analisar agora alguns dos termos e conceitos adotados pela API do *Open MPI*.

Quadro 1 – Principais conceitos adotados pela API do *Open MPI*

| Conceito                   | Explicação                                                                                                                                                                                          |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Processo</b>            | O módulo que executa o seu código. Podemos ter diversos processos rodando em diversas máquinas.                                                                                                     |
| <b>Grupo</b>               | O conjunto formado por todos os processos. Na definição da linguagem, a constante MPI_COMM_WORLD armazena o elemento que gera comunicação entre todos os grupos.                                    |
| <b>Rank</b>                | Seria o identificador de cada processo. Para se distinguir em meio ao grupo, cada processo recebe um <i>rank</i> através da função MPI_Comm que varia entre 0 e N-1, sendo N o número de processos. |
| <b>Comunicador</b>         | O elemento que gera a comunicação entre os processos.                                                                                                                                               |
| <b>Buffer de Aplicação</b> | É o espaço transitório de memória, no qual a leitura e escrita de dados é feita para a comunicação entre os processos. A sua gerência é feita pelo programa.                                        |
| <b>Buffer de Sistema</b>   | Este é o espaço de memória que o sistema reserva para gerenciar as mensagens.                                                                                                                       |



## 3.2 Primeiro Código em Open MPI

A instalação do Open MPI funciona de forma análoga a outras bibliotecas padrão. No site oficial<sup>1</sup> são apresentadas diversas opções de versões para diferentes sistemas. Para usuários do *Visual Studio*, uma implementação própria da Microsoft chamada MS - MPI também está disponível<sup>2</sup>.

Os códigos que discutiremos aqui vale para as principais implementações de MPI sem nenhuma mudança no que diz respeito à codificação. A forma que internamente as funcionalidades são implementadas e o bom aproveitamento das especificidades de cada *hardware* que variam dependendo da implementação escolhida.

Para compilar códigos MPI, ao invés de compiladores comuns como o gcc, é recomendado utilizar o *wrapper* do *open MPI* **mpicc** (para linguagem C). Um *wrapper* é uma espécie de “envelope” do compilador que configura diretivas e bibliotecas antes de compilar o código. Em linux e Mac OS, a compilação seria assim:

```
> mpicc meu_codigo_mpi.c -o meu_programa_mpi
```

Enquanto em Windows, especificamente no *Visual Studio*, dependerá de configurações específicas que podem ser encontradas nos fóruns do ms-mpi da própria Microsoft.

Abaixo temos o código que irá disparar uma determinada quantidade de processos, dependendo do número descrito nos arquivos de configuração.

```
1 #include <stdio.h> // Bibliotecas
2 #include <mpi.h>
3
4 int main() {
5     // Inicializa região paralela
6     MPI_Init(NULL, NULL);
7     // Descobre o número de processos
8     int total_processos;
9     MPI_Comm_size(MPI_COMM_WORLD, &total_processos);
10    // Recupera o seu rank
11    int rank;
12    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
13    // Descobre o nome do processador
14    char nome_processador[MPI_MAX_PROCESSOR_NAME];
15    int tamanho_nome;
```

---

<sup>1</sup>Para saber mais, acesse: <<https://www.open-mpi.org>>. Acesso em: 4 nov. 2019.

<sup>2</sup>Para saber mais, acesse: <<https://docs.microsoft.com/en-us/message-passing-interface/microsoft-mpi>>. Acesso em: 4 nov. 2019.



```
        MPI_Get_processor_name(nome_processador,  
17 &tamanho_nome);  
18        // Mensagem  
19        printf("Processador: %s; Rank: %d; Total: %d\\n",  
20               nome_processador, rank, total_processos);  
21        // Finaliza região paralela.  
22        MPI_Finalize();  
    }
```

Vamos analisar cada linha:

#### **#include <mpi.h> (linha 2)**

Inclui a biblioteca com todas as funções do MPI.

#### **MPI\_Init(NULL, NULL); (linha 6)**

Inicializa a região paralela do MPI. O número de processos disparados depende do padrão definido no arquivo de configuração. É possível trocar os parâmetros NULL para configurar uma quantidade específica de processos. Uma prática comum é colocar os parâmetros de inicialização como entrada **MPI\_Init(argc, argv);** Desta forma, se executado o programa com os parâmetros: > **meu\_programa\_mpi n 4** o programa será disparado com 4 processos, bastando alterar os parâmetros no momento da execução para ter outros valores sem necessidade de recompilar.

#### **MPI\_Comm\_size(MPI\_COMM\_WORLD, &total\_processos); (linha 9)**

Este comando descobre a quantidade de processos do grupo, a quantidade real que o sistema operacional executou.

#### **MPI\_Comm\_rank(MPI\_COMM\_WORLD, &rank); (linha 12)**

Este comando recupera o rank de cada processo, valor entre 0 e o tamanho do grupo -1. Este comando é importante para distinguir unicamente cada processo e, dessa forma, por exemplo, distribuir uma diferente região dos dados para um processo paralelo.

#### **MPI\_Get\_processor\_name(nome\_processador, &tamanho\_nome)(linha 16)**

Este comando recupera o nome do processador no qual se está executando. Ele também devolve o tamanho do nome, que será no máximo o tamanho da constante interna **MPI\_MAX\_PROCESSOR\_NAME**. Descobrir esta informação pode ser útil para distinguir processos. Lembrando que o OpenMPI pode ser executado em **cluster** e sistemas **NUMA** compostos por uma mistura heterogênea de máquinas. Processos que estão sendo executados em máquinas de determinado modelo podem ser melhores para uma determinada



atividade X, enquanto outras máquinas são melhores executando uma atividade Y, tornando importante distinguir estes processadores.

#### **MPI\_Finalize(); (linha 21)**

Este comando encerra a região paralela e o código continua executando no processo inicial que disparou os demais com a função MPI\_INIT()

### **TEMA 4 - COMUNICAÇÃO OPEN MPI**

A troca de mensagens entre diferentes processos é um dos conceitos mais importantes dentro do Open MPI, ela é feita ponto-a-ponto ou de forma coletiva. Ela é importante especialmente para sincronismo e troca de dados.

Tratando da comunicação ponto-a-ponto, geralmente ela é realizada através dos métodos MPI\_Send e MPI\_Recv. A comunicação através destes métodos é síncrona, o que significa que após o envio de uma mensagem o processo não executa mais nada até que seja identificada a confirmação de recebimento. Alternativamente temos também os comandos MPI\_Isend e MPI\_Irecv que, por sua vez, são assíncronos. Como regra geral, o sincronismo compromete o desempenho, então deve ser utilizado somente quando estritamente necessário. Abaixo seguem os parâmetros das duas funções, embora sejam muitos, são fáceis de entender. As versões síncronas recebem os mesmos parâmetros, exceto pelo último **MPI\_Request\***, que justamente serve para comunicar a situação da comunicação se foi concluída com sucesso, falha ou ainda está em andamento.

```
MPI_Send(  
    void* dados,                //0 dado a ser enviado  
    int contagem,              //Quantos dados  
    MPI_Datatype tipo,         //0 tipo do dado  
    int destino,               //Rank do destino  
    int tag,                   //Identificador da mensagem  
    MPI_Comm comunicador,      //Comunicador, mais detalhes a seguir  
    MPI_Request* pedido)       //Comunica status da comunicação  
  
MPI_Irecv(  
    void* dados,                //0 dado  
    int contagem,              //Quantos dados
```



```
MPI_Datatype tipo,      //O tipo do dado
int origem,             //Rank da origem
int tag,                //Identificador da mensagem
MPI_Comm comunicador,   //Comunicador, mais detalhes a seguir
MPI_Status* estado,     //metadados sobre a mensagem
MPI_Request* pedido)    //comunica status da comunicação
```

O comunicador presente nas funções de receber e enviar mensagens representa um agrupamento de processos. A constante **MPI\_COMM\_WORLD** armazena o comunicador padrão que comunica todo o grupo, no entanto é possível criar o próprio comunicador.

Para aumentar o nível de abstração e permitir que processos funcionando em sistemas diferentes se comuniquem, o MPI oferece versões próprias das principais primitivas das linguagens de programação, conforme a quadro abaixo:

Quadro 2 – Equivalência dos tipos MPI em relação à linguagem C

| Tipo MPI           | Equivalente C      |
|--------------------|--------------------|
| MPI_SHORT          | short int          |
| MPI_INT            | int                |
| MPI_LONG           | long int           |
| MPI_LONG_LONG      | long long int      |
| MPI_UNSIGNED_CHAR  | unsigned char      |
| MPI_UNSIGNED_SHORT | unsigned short int |
| MPI_UNSIGNED       | unsigned int       |
| MPI_UNSIGNED_LONG  | unsigned long int  |



|                        |                        |
|------------------------|------------------------|
| MPI_UNSIGNED_LONG_LONG | unsigned long long int |
| MPI_FLOAT              | float                  |
| MPI_DOUBLE             | double                 |
| MPI_LONG_DOUBLE        | long double            |
| MPI_BYTE               | char                   |

Abaixo temos um código simples que realiza a troca de mensagens entre um processo de rank 0 com outro processo de rank 1.

```
1 int rank;
2 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
3 int tamanho;
4 MPI_Comm_size(MPI_COMM_WORLD, &tamanho);
5
6 int numero;
7 if (rank == 0) {
8     numero = -1;
9     MPI_Send(&numero, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
10 } else if (rank == 1) {
11     MPI_Recv(&numero, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
12             MPI_STATUS_IGNORE);
13     printf("Processo 1; recebeu o numero %d do processo 0\n",
14           numero);
15 }
```

No código, vemos na linha 7 um **if** verificando qual o rank do processo executando; na linha 9, o processo de rank 0 envia o número -1 como mensagem para o processo de rank 1, o qual é recebido como comando `MPI_Recv` da linha 11.

O código envia mensagens ponto-a-ponto, no entanto é possível enviar uma mensagem para todos os processos do grupo simultaneamente, através do comando `MPI_Bcast`. Os parâmetros são os mesmos.

```
int MPI_Bcast(void *dados, int contagem, MPI_Datatype tipo,int origem,
MPI_Comm comunicador);
```



## 4.1 Sincronismo

O *Open MPI* possui dois comandos importantes, relacionados ao sincronismo das atividades. Por vezes é interessante que um processo aguarde a conclusão de um determinado trecho de código por parte de outro processo para continuar a executar. Nesta situação existe o comando `MPI_Wait`.

```
int MPI_Wait(MPI_Request *pedido, MPI_Status *estado);
```

O **`MPI_Request*`**, como já vimos, é associado a vários comandos assíncronos como o `MPI_Irecv` e `MPI_Isend`.

Outro comando importante é o `MPI_Barrier`, para garantir um ponto de sincronismo entre todos os processos que fazem parte do grupo. Enquanto todos os processos não chegam no ponto em questão, os demais aguardam.

```
int MPI_Barrier(MPI_Comm comunicador);
```

## TEMA 5 – DISTRIBUIÇÃO DE TRABALHO

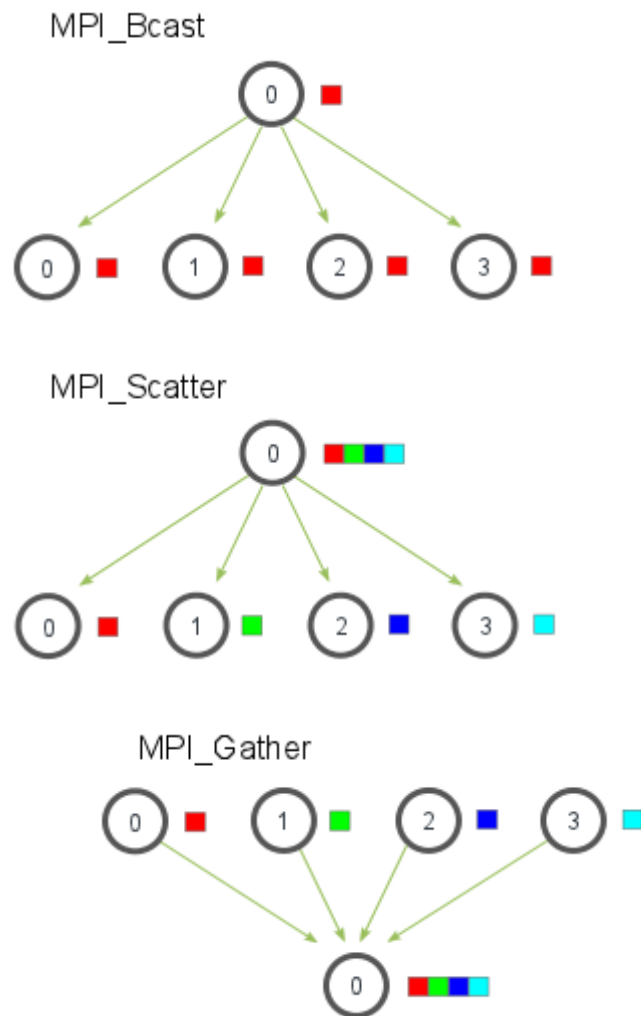
Dois conceitos importantes quando falamos de programação paralela são chamados de *scatter* (espalhar) e *gather* (coletar). Como os nomes sugerem a técnica de *scatter*, consiste em espalhar um conjunto de dados entre diversos processos, sendo que cada um trabalhará no conjunto de dados que recebeu e posteriormente uma função *gather* será executada para reunir as repostas.

***Scatter***: No *Open MPI* a função `MPI_Scatter` implementa o conceito de *scatter* de forma automática. A ideia é semelhante ao `MPI_Bcast` que manda o mesmo dado para todos os processos. E o `MPI_Scatter` divide cada trecho do dado.

As figuras abaixo ilustram a divisão das mensagens com as funções `MPI_Bcast`, `MPI_Scatter` e `MPI_Gather`



Figura 3 - Divisão das mensagens com as funções MPI\_Bcast, MPI\_Scatter e MPI\_Gather



Fonte: <<https://mpitutorial.com/tutorials/mpi-scatter-gather-and-allgather/>>. Acesso em: 5 nov. 2019.

```
MPI_Scatter(  
    void* dados,    //o conjunto total dos dados  
    int contagem,   //a quantidade de dados  
    MPI_Datatype tipo,    //o tipo dos dados  
    void* dados_individual,    //a parte dos dados de cada processo  
    int total_individual, //a quantidade de dados que cada processo recebe  
    MPI_Datatype tipo_individual, //o tipo dos dados individuais  
    int origem, //rank do processo de origem  
    MPI_Comm comunicador) //o comunicador
```





**Gather:** No *Open MPI*, o comando `MPI_Gather` reúne os dados divididos entre os processos pelo comando `MPI_Scatter`. Abaixo seguem os parâmetros de função.

```
MPI_Gather(  
    void* dados, //o conjunto total dos dados  
    int contagem, //a quantidade de dados  
    MPI_Datatype tipo, //o tipo dos dados  
    void* dados_individual, //a parte dos dados de cada processo  
    int total_individual, //rank do processo de origem  
    MPI_Datatype tipo_individual, //o tipo dos dados individuais  
    int origem, //rank do processo de origem  
    MPI_Comm comunicador) //o comunicador
```

Na sequência, vemos um código que ilustra o conceito de *scatter/gather*, no qual tiramos a média de um vetor e, para essa média, é necessário realizar duas atividades: somar todos os valores e, na sequência, dividir o somatório pelo total de itens. Quanto maior o número de itens, maior será o número de instruções. Este tipo de código é o cenário perfeito para *scatter/gather* por permitir dividirmos o trabalho de realizar a média entre vários processos e posteriormente unirmos os seus resultados.

Em linhas gerais, a ideia do código está em dividir entre os vários processos o array **numeros** através da função *Scatter* (linha 10). Na sequência, cada processo irá calcular a média da sua parcela dos dados (linha 13) e, via função *Gather* (linha 19), reunir novamente os dados. Quando o processo de rank igual a 0 chegar, a linha 23 é a média final entre os resultados parciais de cada processo.

```
1 // Array com números aleatórios criado pelo processo de rank 0  
2 if (rank == 0) {  
3     numeros = numeros_aleatorios(elementos_por_proc * total_proc);  
4 }  
5  
6 // Criação de buffer para dividir array de números aleatórios  
7 float *sub_numeros = malloc(sizeof(float) * elementos_por_proc);  
8  
9 // Operação Scatter do array entre os processos  
10
```



```
    MPI_Scatter(numeros, elementos_por_proc, MPI_FLOAT, sub_numeros,
11 elementos_por_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);
12
13 // computar media do sub_conjunto dos numeros
    float sub_media = computar_media(sub_numeros,
14 elementos_por_proc);
15 // reunir as médias parciais no processo de rank 0
16 float *sub_medias = NULL;
17 if (rank == 0) {
18     sub_medias = malloc(sizeof(float) * total_proc);
19 }
    MPI_Gather(&sub_media, 1, MPI_FLOAT, sub_medias, 1, MPI_FLOAT, 0,
20 MPI_COMM_WORLD);
21
22 // Computar a média final.
23 if (rank == 0) {
24     float avg = computar_media(sub_medias, total_proc);
    }
}
```

## FINALIZANDO

Nesta aula aprendemos sobre computação paralela em um ambiente distribuído. Discutimos as formas de trabalhar com *Open MPI* e duas estratégias muito importantes dentro da programação paralela. Os comandos *Scatter* e *Gather*, que resolvem uma gama muito grande de problemas, nos quais se deseja dividir uma grande quantidade de dados para ser individualmente processados e depois reunidos em um mesmo conjunto de dados.



## REFERÊNCIAS

- BRESHEARS, C. **The art of concurrency**. O'Reilly, 2009.
- CHANDRA, R. et al. **Parallel programming in Open MP**. 1. ed. Morgan-Kaufmann: 2001.
- CHAPMAN, B.; JOST, G.; VAN DER PAS, R. **Using OpenMP - portable shared memory parallel programming**. 1. ed. MIT Press, 2008.
- DANTAS, M. **Computação distribuída de alto desempenho**. 1. ed. Axcel, 2005.
- DE ROSE, C. A. F.; NAVAUX, P. O. A. **Arquiteturas paralelas**. 1. ed. Bookman, 2005.
- KIRK, D. B.; HWO, W. W. **Programming massively parallel processors – a hands-on approach**. 1. ed. Morgan-Kaufmann, 2010.
- KUMAR, V.; KARYPIS, G.; GUPTA, A. **Introduction to parallel computing**. 2. ed. Pearson, 2003.
- PACHECO, P. S. **An introduction to parallel programming**. 1. ed. Morgan Kaufmann, 2011.
- RAUBER, T.; RÜNGER, G. **Parallel programming: for multicore and cluster systems**. 1. ed. Springer, 2010.
- STALLINGS, W. **Arquitetura e organização de computadores**. 10. ed. Pearson, 2017.